Exercise 1

Sound Classification Output & Visualization Sketch

Music and Multimedia Streaming Lab

README — Processing + Wekinator

Color Mapping — Current Experiment

State	Sound	Canvas color
Class 1	`eeeeeeeeeeeee`	Red `(255, 0, 0)`
Class 2	`uuuuuuuuuuuuu`	Blue `(0, 0, 255)`
Waiting / unknown	—	Dark gray `(80)`

OSC Settings

Listening port	`12000`
OSC address	`/wek/outputs`
Expected data type	one `float`
Expected values	`1.0` or `2.0`

Contents

1 Purpose of This Report **3**

2 Big Picture of Exercise 1 **3**

3 What This Sketch Is **3**

4 What This Sketch Does as a Standalone Program **4**

5 Complete Code **4**

6 Block-by-Block Code Explanation **5**
6.1 Header Comments . . . 5
6.2 Importing the OSC Library . . . 5
6.3 Global Variable Declarations . . . 6

7 The `setup()` Function **6**
7.1 Window Size . . . 6
7.2 Starting the OSC Listener . . . 6
7.3 Text Alignment and Size . . . 7

8 The `draw()` Function **7**
8.1 Color Logic . . . 7
8.2 Drawing the Text . . . 8

9 The `oscEvent()` Function **8**
9.1 Checking the OSC Address . . . 8
9.2 Checking the Data Type . . . 8
9.3 Reading and Converting the Output Value . . . 9
9.4 Updating the Message and Console . . . 9

10 How the Three Functions Work Together **9**

11 OSC Communication Details **10**

12 Why One Output for Two Classes **10**

13 Lab Requirement Checklist **10**

14 Current Project Configuration **11**

15 How to Run the Complete System **11**

16 Why This Sketch Does Not Use Sound Libraries **11**

17 Debugging Checklist **12**
17.1 Screen Stays Gray — “Waiting for Wekinator...” . . . 12
17.2 Console Shows Nothing . . . 12
17.3 Wrong Color Appears . . . 12
17.4 Color Flickers Rapidly . . . 12

18 Possible Improvements **13**
18.1 Add Sound Names to the Label . . . 13

18.2 Smooth Color Transitions 13
18.3 Support More Classes 13

19 Discussion Points for the Final Report **13**

20 Summary of the Code **13**

21 Final Conclusion **14**

1. Purpose of This Report

This README explains the Processing **output sketch** used for Exercise 1 — Sound Classification in the Music and Multimedia Streaming lab.

The purpose of this Processing sketch is to receive the predicted class from Wekinator and visualize the result by changing the canvas color in real time.

This report explains:

1. What this sketch does in the complete project.
2. What this sketch does as a standalone Processing program.
3. The complete code.
4. Each line and each code block in detail.
5. How OSC communication works between Wekinator and this sketch.
6. Why this sketch is the **output sketch**, not the input sketch.
7. How this sketch fulfills the visualization requirement of the lab.
8. How to run and test this sketch correctly.
9. Common problems and debugging advice.
10. Discussion points for the final lab report.

2. Big Picture of Exercise 1

Exercise 1 builds a real-time sound classification system. The complete pipeline is:

```
Microphone
↓
Processing input sketch
↓
Audio feature extraction using FFT
↓
OSC message:  /wek/inputs
↓
Wekinator classifier
↓
OSC message:  /wek/outputs
↓
Processing output sketch (this file)
↓
Canvas color changes according to the predicted sound class
```

This README focuses exclusively on the **last part** of the pipeline:

```
Wekinator prediction → Processing receives output → canvas color changes
```

3. What This Sketch Is

This file is the **output / visualization sketch**.

- Does **not** listen to the microphone.
- Does **not** compute FFT.
- Does **not** train the classifier.
- Does **not** send input features to Wekinator.

Instead, it waits for Wekinator to send a classification result and then changes the screen color:

State	Sound class	Canvas color
Class 1	`eeeeeeeeeeeee`	Red
Class 2	`uuuuuuuuuuuuu`	Blue
Waiting / unknown	(none received yet)	Dark gray

So the full interaction flow is:

- Say `eeeeeeeeeeeee` → Wekinator predicts Class 1 → output sketch becomes **red**.
- Say `uuuuuuuuuuuuu` → Wekinator predicts Class 2 → output sketch becomes **blue**.

4. What This Sketch Does as a Standalone Program

Even outside the lab project, this Processing program can be understood as a general-purpose OSC receiver. As a standalone program, it:

1. Opens a 400 × 400 Processing window.
2. Starts listening for OSC messages on port `12000`.
3. Waits for messages with the OSC address `/wek/outputs`.
4. Reads the first value from the OSC message.
5. Converts that value into an integer class number.
6. Stores the class number in `predictedClass`.
7. Changes the background color based on that class number.
8. Displays the class label centered on screen.
9. Prints the received class to the Processing console.

This sketch is not limited to Wekinator. It is a general example of how Processing can receive OSC messages and use them to control graphics in real time.

5. Complete Code

```
// Exercise 1 - Output / Visualization Sketch
// Receives one classifier output from Wekinator and changes
    canvas color.

import oscP5.*;

```

```
OscP5 oscP5;
int predictedClass = 0;
String currentMessage = "Waiting for Wekinator...";

void setup() {
  size(400, 400);
  oscP5 = new OscP5(this, 12000); // Wekinator sends outputs to port 12000 by default
  textAlign(CENTER, CENTER);
  textSize(32);
}

void draw() {
  if (predictedClass == 1) {
    background(255, 0, 0);    // Class 1 = red
  } else if (predictedClass == 2) {
    background(0, 0, 255);    // Class 2 = blue
  } else {
    background(80);           // Waiting / unknown
  }

  fill(255);
  text(currentMessage, width / 2, height / 2);
}

void oscEvent(OscMessage theOscMessage) {
  if (theOscMessage.checkAddrPattern("/wek/outputs")) {
    if (theOscMessage.checkTypetag("f")) {
      float outputValue = theOscMessage.get(0).floatValue();
      predictedClass    = int(outputValue);
      currentMessage    = "Class " + predictedClass;
      println("Received class: " + predictedClass);
    }
  }
}
```

Listing 1: Full output / visualization sketch

6. Block-by-Block Code Explanation

6.1 Header Comments

```
// Exercise 1 - Output / Visualization Sketch
// Receives one classifier output from Wekinator and changes canvas color.
```

Comments are ignored by Processing at runtime; they are written for human readers. These two lines clearly identify the sketch as the *output* side of the project — it receives predictions, it does not produce features.

6.2 Importing the OSC Library

```
import oscP5.*;
```

This is the only import needed. Compared to the input sketch, notice what is absent:

Library	Input sketch	Output sketch
`processing.sound.*`	Required (microphone, FFT)	Not needed
`oscP5.*`	Required (send messages)	Required (receive messages)
`netP5.*`	Required (define destination)	Not needed

The output sketch only listens — it never sends — so `netP5` and the sound library are unnecessary.

6.3 Global Variable Declarations

```
OscP5 oscP5;
int predictedClass = 0;
String currentMessage = "Waiting for Wekinator...";
```

Variable	Type	Purpose
`oscP5`	`OscP5`	Manages all OSC communication. Initialized in `setup()`.
`predictedClass`	`int`	Stores the last class received. Starts at `0` (no class yet).
`currentMessage`	`String`	Text shown on screen. Starts as a waiting message.

`0` is not one of the trained sound classes. It is the default value before any prediction arrives, and it triggers the gray "waiting" background.

7. The `setup()` Function

```
void setup() {
  size(400, 400);
  oscP5 = new OscP5(this, 12000);
  textAlign(CENTER, CENTER);
  textSize(32);
}
```

`setup()` runs **once** when the sketch starts. It initializes the window, starts the OSC listener, and sets text styling.

7.1 Window Size

```
size(400, 400);
```

Creates a 400 × 400 pixel canvas. The sketch only needs to display a background color and a text label, so a compact square window is enough.

7.2 Starting the OSC Listener

```
oscP5 = new OscP5(this, 12000);
```

This is one of the most important lines. It opens an OSC receiver on port `12000`.

If Wekinator is configured to send outputs to a different port, this sketch will receive nothing. Always verify that the Wekinator output port equals `12000`.

The required Wekinator output configuration is:

```
Output OSC message : /wek/outputs
Host               : localhost
Port               : 12000
Number of outputs  : 1
Type               : classifier
Number of classes  : 2
```

7.3 Text Alignment and Size

```
textAlign(CENTER, CENTER);
textSize(32);
```

`textAlign(CENTER, CENTER)` makes text drawn at a point appear visually centered around that point — both horizontally and vertically. Combined with drawing at `(width/2, height/2)`, this places the message exactly in the middle of the window.

8. The `draw()` Function

```
void draw() {
  if (predictedClass == 1) {
    background(255, 0, 0);    // Class 1 = red
  } else if (predictedClass == 2) {
    background(0, 0, 255);    // Class 2 = blue
  } else {
    background(80);           // Waiting / unknown
  }

  fill(255);
  text(currentMessage, width / 2, height / 2);
}
```

`draw()` runs **continuously** — many times per second. Each frame, it reads the current `predictedClass` and repaints the canvas accordingly.

8.1 Color Logic

predictedClass	Condition	RGB call	Result
1	`== 1`	`background(255, 0, 0)`	Pure red
2	`== 2`	`background(0, 0, 255)`	Pure blue
0	`else`	`background(80)`	Dark gray

`background(R, G, B)` uses values from 0–255. A single value like `background(80)` sets a grayscale tone.

8.2 Drawing the Text

```
fill(255);
text(currentMessage, width / 2, height / 2);
```

`fill(255)` sets the text color to white — visible on all three background colors (red, blue, gray). The text is drawn at the canvas center:

$$\texttt{width/2} = 200 \quad \texttt{height/2} = 200$$

The three possible messages are:

```
"Waiting for Wekinator..."   (initial state, predictedClass = 0)
"Class 1"                    (after receiving class 1)
"Class 2"                    (after receiving class 2)
```

9. The `oscEvent()` Function

```
void oscEvent(OscMessage theOscMessage) {
  if (theOscMessage.checkAddrPattern("/wek/outputs")) {
    if (theOscMessage.checkTypetag("f")) {
      float outputValue = theOscMessage.get(0).floatValue();
      predictedClass    = int(outputValue);
      currentMessage    = "Class " + predictedClass;
      println("Received class: " + predictedClass);
    }
  }
}
```

`oscEvent()` is a **callback function** provided by the oscP5 library. It is called automatically whenever an OSC message arrives — we never call it manually.

9.1 Checking the OSC Address

```
if (theOscMessage.checkAddrPattern("/wek/outputs")) {
```

Filters out any OSC messages that do not come from Wekinator's output stream. A Processing sketch can receive messages from multiple sources; this guard ensures only `/wek/outputs` messages are processed.

Address	Direction
`/wek/inputs`	Processing → Wekinator (input sketch sends this)
`/wek/outputs`	Wekinator → Processing (this sketch listens for this)

9.2 Checking the Data Type

```
if (theOscMessage.checkTypetag("f")) {
```

The typetag `"f"` means one floating-point number. Wekinator sends classifier outputs as floats even when they represent integer class labels:

```
Class 1  ->  Wekinator sends  /wek/outputs 1.0
Class 2  ->  Wekinator sends  /wek/outputs 2.0
```

This check avoids errors if an unexpected message type is received.

9.3 Reading and Converting the Output Value

```
float outputValue = theOscMessage.get(0).floatValue();
predictedClass    = int(outputValue);
```

`get(0)` retrieves the first (and only) value in the message. `int(outputValue)` converts the float to an integer:

$$1.0 \rightarrow 1 \qquad 2.0 \rightarrow 2$$

The integer is then used in `draw()` to select the background color.

9.4 Updating the Message and Console

```
currentMessage = "Class " + predictedClass;
println("Received class: " + predictedClass);
```

The `+` operator concatenates text. If `predictedClass = 1`, `currentMessage` becomes `"Class 1"`. The `println()` writes to the Processing console — useful for debugging when the visual output is not behaving as expected.

10. How the Three Functions Work Together

The complete event flow for each class prediction:

```
Class 1 flow:
Wekinator sends /wek/outputs 1.0
↓
oscEvent() is called automatically
↓
predictedClass ← 1
↓
currentMessage ← "Class 1"
↓
draw() sees predictedClass == 1
↓
background(255, 0, 0) - canvas turns red
```

```
Class 2 flow:
Wekinator sends /wek/outputs 2.0
↓
oscEvent() is called automatically
↓
predictedClass ← 2
↓
currentMessage ← "Class 2"
↓
draw() sees predictedClass == 2
↓
background(0, 0, 255) - canvas turns blue
```

11. OSC Communication Details

Setting	Value
Processing listens on	port `12000`
Expected OSC address	`/wek/outputs`
Expected data type	one `float` (`"f"`)
Expected values	`1.0` (class 1) or `2.0` (class 2)

The relationship between the three parts of the system:

```
Input sketch ──/wek/inputs port 6448──> Wekinator ──/wek/outputs port 12000──> Output sketch
```

12. Why One Output for Two Classes

Wekinator uses `# outputs = 1` with `number of classes = 2`. The single output value encodes the predicted class:

Wekinator output value	Meaning
`1.0`	Class 1 (`eeeee`)
`2.0`	Class 2 (`uuuuu`)

One output is sufficient to represent a multi-class label in Wekinator's classification mode. This sketch converts that single float into an integer and selects the matching canvas color.

13. Lab Requirement Checklist

Lab requirement	Code that fulfills it
Receive OSC from Wekinator	`new OscP5(this, 12000)`
Listen for Wekinator output address	`checkAddrPattern("/wek/outputs")`
Receive one classifier output	`checkTypetag("f") + get(0).floatValue()`
Store predicted class	`predictedClass = int(outputValue)`
Class 1 visual response	`background(255, 0, 0)`
Class 2 visual response	`background(0, 0, 255)`
Waiting / unknown visual state	`background(80)`
Display class label on screen	`text(currentMessage, width/2, height/2)`
Debug output to console	`println("Received class:  " + predictedClass)`

14. Current Project Configuration

Completed Experiment Configuration

Input feature representation	12 reduced FFT bands
Input sketch sends	12 inputs
Wekinator input message	`/wek/inputs`
Wekinator input port	`6448`
Wekinator output message	`/wek/outputs`
Wekinator output port	`12000`
Wekinator outputs	`1`
Output type	classification
Number of classes	2
Classifier	K-Nearest Neighbor, $k = 3$
Class 1 sound	`eeeeeeeeeeeee` → red screen
Class 2 sound	`uuuuuuuuuuuuu` → blue screen

15. How to Run the Complete System

1. **Run the input Processing sketch.** Set `featureMode = 3` (12 FFT features). The input sketch must run because Wekinator does not access the microphone directly.
2. **Open the trained Wekinator project.** Verify: 12 inputs, 1 output, 2 classes, KNN with $k = 3$.
3. **Click Run in Wekinator.** Wekinator begins classifying live features from the input sketch.
4. **Run this output sketch.** It opens a 400×400 window and starts listening on port `12000`.
5. **Test the sounds:**

   Say `eeeeeeeeeeeee` → Expected: canvas turns **red**, text shows **Class 1**.

   Say `uuuuuuuuuuuuu` → Expected: canvas turns **blue**, text shows **Class 2**.

During **training**, only the input sketch and Wekinator need to run. This output sketch is not required during the recording of examples.

During **final testing and demonstration**, all three parts must run simultaneously: *Input sketch + Wekinator (Run mode) + Output sketch*

16. Why This Sketch Does Not Use Sound Libraries

This sketch does not import `processing.sound.*` because audio analysis is entirely handled by the input sketch.

The separation of responsibilities is intentional:

```
Input sketch   →  audio analysis and feature sending
Wekinator   →  machine learning classification
Output sketch   →  visual response only
```

This design makes each component easier to understand, debug, and present independently.

17. Debugging Checklist

17.1 Screen Stays Gray — "Waiting for Wekinator..."

The sketch has not received a valid class message. Check:

- Is Wekinator open and trained?
- Did you click **Run** in Wekinator (not just Train)?
- Is the input sketch running?
- Is Wekinator receiving inputs from the input sketch?
- Is the Wekinator output port set to `12000`?
- Is the Wekinator output message set to `/wek/outputs`?
- Is this Processing sketch listening on port `12000`?

17.2 Console Shows Nothing

No valid `/wek/outputs` message is arriving. Most likely causes:

- Wekinator is not in **Run** mode.
- Wekinator is sending to the wrong port.
- The OSC message address does not match `/wek/outputs`.
- Another program is already using port `12000`.

17.3 Wrong Color Appears

If class 1 shows blue or class 2 shows red, check the mapping in `draw()`. To swap the colors, change:

```
background(255, 0, 0);    // currently Class 1 = red
background(0, 0, 255);    // currently Class 2 = blue
```

17.4 Color Flickers Rapidly

Wekinator may be uncertain between the two classes. Solutions:

- Record more training examples.
- Use clearly distinct sounds.
- Reduce background noise.
- Keep the microphone distance consistent.
- Try a different feature mode in the input sketch.

18. Possible Improvements

18.1 Add Sound Names to the Label

Instead of displaying only `"Class 1"`, show the associated sound:

```
if (predictedClass == 1) {
  currentMessage = "Class 1: eeeee";
} else if (predictedClass == 2) {
  currentMessage = "Class 2: uuuuu";
}
```

18.2 Smooth Color Transitions

Instead of instantly switching color, the sketch could interpolate (fade) between colors using `lerp()` on the RGB values. This would make the visualization more polished, though the lab only requires a simple color change.

18.3 Support More Classes

The conditional logic can be extended for additional classes:

```
// Example extension for 4 classes
if      (predictedClass == 1) background(255, 0,   0);   // red
else if (predictedClass == 2) background(0,   0, 255);   // blue
else if (predictedClass == 3) background(0, 200,   0);   // green
else if (predictedClass == 4) background(255, 200, 0);   // yellow
else                          background(80);
```

Exercise 1 requires two classes, so the current two-class version is correct for the lab.

19. Discussion Points for the Final Report

This output sketch is simple, but it is important because it demonstrates a complete real-time interactive loop:

```
human sound → audio features → KNN classification → visual feedback
```

Key points worth discussing in the final report:

- The visual response makes the classifier immediately interpretable — the user does not need to look at Wekinator's internal interface.
- The sketch demonstrates that Wekinator can control an external application in real time through OSC.
- Processing handles interaction and visualization; Wekinator handles machine learning. Together they form a modular intelligent system.
- The color change completes the feedback loop: the human makes a sound and receives an immediate, unambiguous visual response.

20. Summary of the Code

The entire sketch can be summarized in six steps:

```
1.  Open a 400 x 400 window
2.  Listen for Wekinator output messages on port 12000
3.  If received class == 1 -> canvas turns red
4.  If received class == 2 -> canvas turns blue
5.  If no class received -> canvas stays dark gray
6.  Display current class as centered white text
```

The six most important lines:

```
oscP5 = new OscP5(this, 12000);                          // start OSC listener
theOscMessage.checkAddrPattern("/wek/outputs")           // accept only Wekinator outputs
theOscMessage.get(0).floatValue()                        // read the predicted class
predictedClass = int(outputValue);                       // convert float to integer
background(255, 0, 0);                                   // class 1 = red
background(0, 0, 255);                                   // class 2 = blue
```

21. Final Conclusion

This Processing sketch correctly fulfills the output visualization requirement of Exercise 1. It receives Wekinator's predicted sound class through OSC and changes the canvas color in real time according to the prediction.

Key Properties of this Sketch

- OSC listener on port `12000`
- Reacts only to `/wek/outputs` messages
- Reads one `float` per message and converts to integer class
- Class 1 → red canvas
- Class 2 → blue canvas
- Waiting / unknown → dark gray canvas
- Centered white text label on all backgrounds
- Console debug output for every received class

The final system, when all three components run together, demonstrates the full interactive pipeline:

```
live sound input   ↓
FFT feature extraction (input sketch)   ↓
KNN classification in Wekinator   ↓
real-time color visualization (this sketch)
```

This output sketch is therefore the correct and complete second Processing project for the Exercise 1 deliverable.